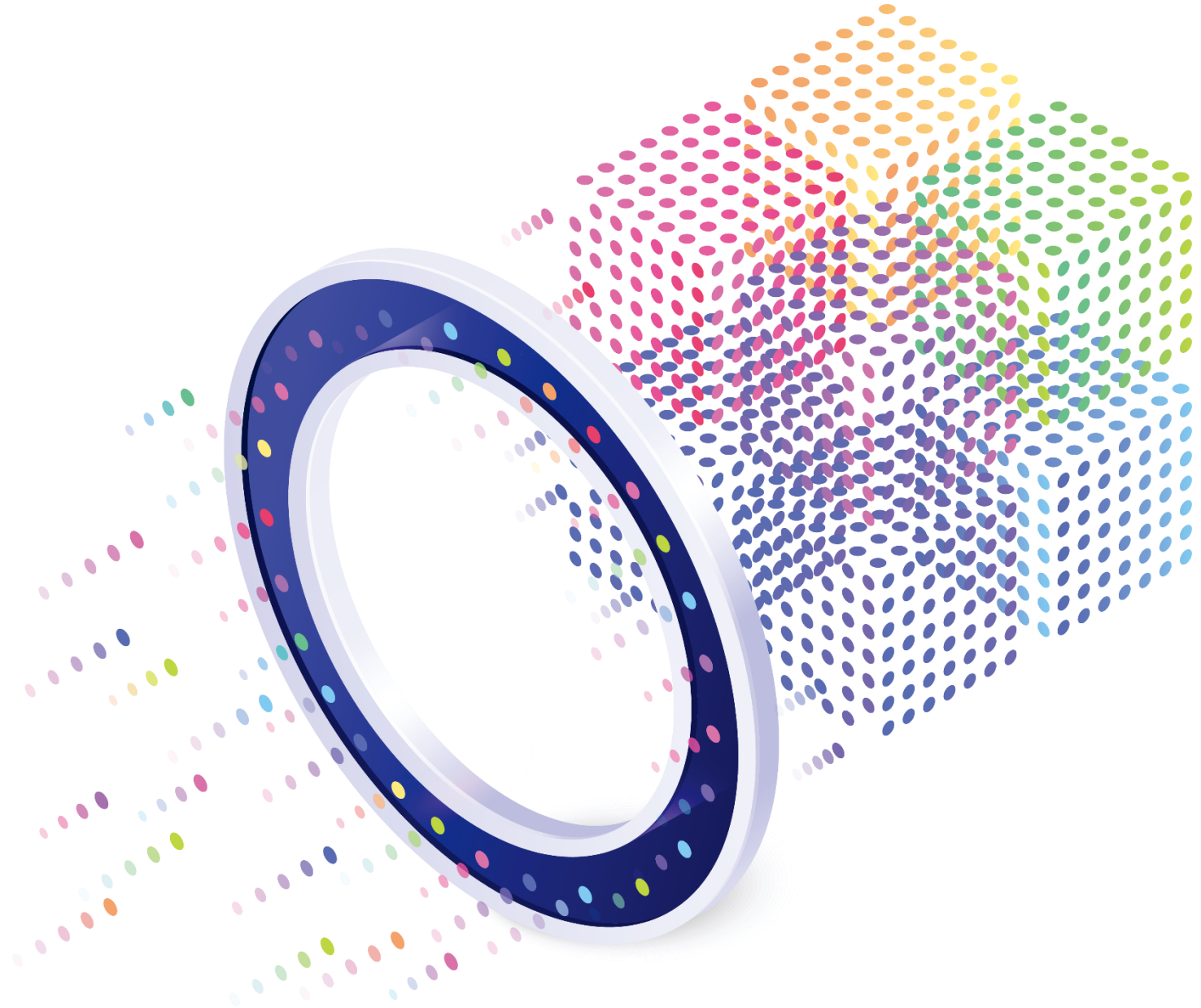


Azure Functions

서버리스 컴퓨팅으로 생산성과
효율성을 극대화



이 자료는 Elixir의 사전 서면 승인 없이 외부에 배포하기 위해 그 일부를 배포, 인용 또는 복제할 수 없습니다.

© Copyright Elixir

Azure Functions: 서버리스 컴퓨팅

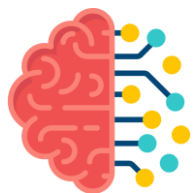
애플리케이션 개발의 새로운 패러다임

서버리스 컴퓨팅으로 개발 생산성과 효율성을 극대화하는 방법

Azure Functions란 무엇일까요?



Azure Functions는 애플리케이션 실행에 필요한 인프라와 리소스를 지속적으로 업데이트하여 제공하는 **주문형(On-Demand) 클라우드 서비스**입니다.



개발자는 핵심 코드에만 집중, 나머지 부분은 Azure Functions가 처리합니다.



Azure 서버리스 컴퓨팅을 대표하는 서비스입니다.



웹 API 구축, 데이터 처리, IoT, 메시지 큐 등 다양한 분야에 활용 가능합니다.



필요할 때만 리소스를 사용, 사용한 만큼만 비용을 지불합니다.

Azure Functions 주요 장점



생산성 향상: 서버 관리 부담 감소, 개발에만 집중!



비용 효율성: 사용한 만큼만 지불하는 합리적인 과금 방식



자동 확장: 트래픽 증가에도 자동으로 리소스 확장



빠른 개발: 개발 및 배포 과정을 간소화, 빠른 개발 속도



다양한 서비스 연동: Azure 서비스는 + 외부 시스템과 쉽게 연결(Azure Storage, Logic Apps, Github, Twilio, Event Hubs, Stream Analytics, Cosmos DB)

Azure Functions 트리거

Azure Function는 스스로 알아서 척척 실행되는 마법 상자가 아닙니다. 우리가 어떤 일을 시작하려고 할 때 '시작' 버튼을 누르거나, 아침에 알람이 울려야 잠에서 깨어나는 것처럼, Azure Functions도 작동을 시작하게 하는 '신호'가 필요합니다. 이 신호 역할을 하는 것이 바로 "트리거(Trigger)" 입니다.

트리거는 마치 **기폭제나 알람 시계**와 같습니다.



폭탄에 불을 붙이는 기폭제처럼, 특정 사건이나 신호가 발생하면 Azure Functions가 작동을 시작합니다.



아침에 울리는 알람처럼, 미리 설정된 시간이 되거나 특정 조건이 충족되면 Azure Functions가 실행됩니다

Azure Functions 트리거 트리거 종류 및 예시(1)

HTTP 트리거:



웹사이트나 앱에서 정보를 요청할 때처럼, 웹 요청(HTTP 요청)이 들어오면 Azure Functions가 '주문 받았 습니다!' 하고 일어나서 일하는 것과 같습니다.



예를 들어, 쇼핑몰에서 '상품 정보 보기' 버튼을 클릭하면, 해당 상품 정보를 가져오는 Azure Functions가 HTTP 트리거를 통해 실행됩니다. 이것은 API 서버를 만들 때 아주 유용합니다.

Timer 트리거:



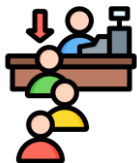
알람 시계처럼, 미리 설정해 놓은 시간이 되면 Azure Functions가 '이제 일할 시간이다!' 하고 작동을 시작 하는 것과 같습니다.



예를 들어, 매일 밤 12시에 자동으로 데이터를 백업하는 Azure Functions를 Timer 트리거로 설정할 수 있습니다.

Azure Functions 트리거 트리거 종류 및 예시(2)

Queue 트리거:



음식점에서 손님이 주문한 음식을 기다리는 번호표처럼, 메시지 큐에 새로운 주문이나 작업이 들어오면 Azure Functions가 '새로운 작업이 도착했습니다!' 하고 반응하는 것과 같습니다.

예를 들어, 온라인 쇼핑몰에서 고객이 상품을 주문하면, 해당 주문 정보를 메시지 큐에 넣고, Queue 트리거를 통해 주문 처리 Azure Functions가 실행됩니다.

Blob Storage 트리거:



사진을 클라우드 저장소에 업로드하면, Azure Functions가 '새로운 사진이 올라왔네!' 하고 인식을 하고, 해당 사진에 썸네일을 자동으로 생성하는 것처럼, Blob Storage에 파일이 업로드 되면 Azure Functions가 동작하는 것과 같습니다.

예를 들어, Blob Storage에 새로운 이미지 파일이 업로드 되면 자동으로 썸네일을 생성하는 Azure Functions를 만들 수 있습니다.

Azure Functions 바인딩

Azure Functions 바인딩은 마치 함수가 데이터를 주고받을 때 사용하는 편리한 "도구 상자" 와 같습니다. 트리거가 함수를 " 깨우는 " 역할을 했다면, 바인딩은 깨어난 함수가 일을 하는 데 필요한 재료(데이터) 를 가져오거나, 일이 끝난 결과물(데이터) 을 내보내는 역할을 합니다.



입력 바인딩 (Input Binding): "재료 가져오기 도구"

- 함수가 실행될 때 필요한 데이터(재료)를 자동으로 가져오는 도구입니다.
- 요리사가 냉장고에서 요리 재료를 꺼내 오는 것처럼, 함수가 외부 서비스(예: 데이터 보관 창고인 Blob Storage, 데이터베이스인 Cosmos DB)에서 데이터를 알아서 꺼내 와서 함수 안에서 바로 쓸 수 있게 해줍니다.
- 예시: Blob Storage 입력 바인딩: "Blob Storage에 있는 사진 파일을 가져와서 이미지 처리 함수에서 사용해 줘!" -> 함수가 사진 파일을 알아서 가져옴



출력 바인딩 (Output Binding): "결과물 내보내기 도구"

- 함수가 일을 끝낸 결과물(데이터)을 자동으로 외부 서비스에 저장하거나 보내주는 도구입니다.
- 요리사가 완성된 요리를 접시에 담아 내놓는 것처럼, 함수가 처리한 결과를 데이터베이스에 저장하거나, 다른 큐에 보내는 일을 알아서 해주는 거예요.
- 예시: Cosmos DB 출력 바인딩: "함수에서 만든 새로운 사용자 정보를 Cosmos DB에 저장해 줘!" -> 함수가 사용자 정보를 알아서 저장

Azure Functions 활용 분야 (1)

Azure Functions를 사용할 수 있는 다양한 시나리오:



파일 업로드 처리: Blob Storage에 파일이 업로드 되면 자동으로 처리합니다.



실시간 데이터 처리: IoT 기기에서 생성되는 데이터를 실시간으로 처리합니다.



AI 서비스와 연동: Azure AI Services를 활용하여 큐에서 가져온 텍스트를 분석하고 감정 분석, 언어 감지 등 분류 작업을 수행합니다.



예약 작업 실행: 정해진 시간 간격으로 데이터 정리 작업을 실행합니다.

Azure Functions 활용 분야 (2)



확장 가능한 API 개발: HTTP 트리거를 사용하여 API 엔드 포인트를 구축합니다.



서버리스 워크플로 구축: Durable Functions를 이용하여 복잡한 워크플로를 구현합니다.



데이터베이스 변경 감지: 데이터베이스 변경이 발생하면 사용자 지정 로직을 실행합니다.

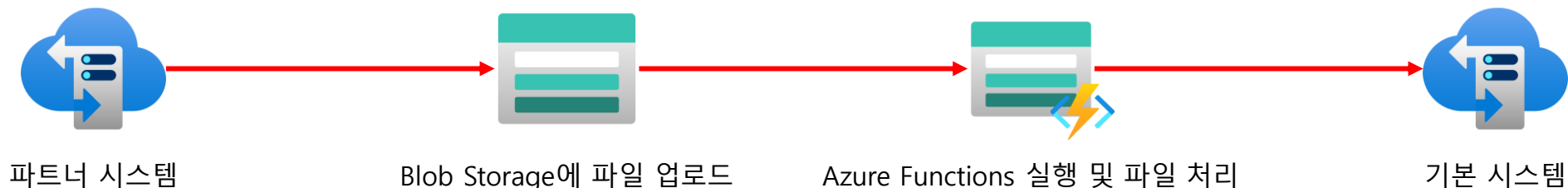


메시지 시스템 처리: 메시지 큐에 저장된 메시지를 처리합니다.

실전 시나리오: 파일 업로드 및 처리 자동화

파트너 시스템에서 제품 카탈로그 정보를 파일 형태로 Blob Storage에 업로드하는 시나리오입니다.

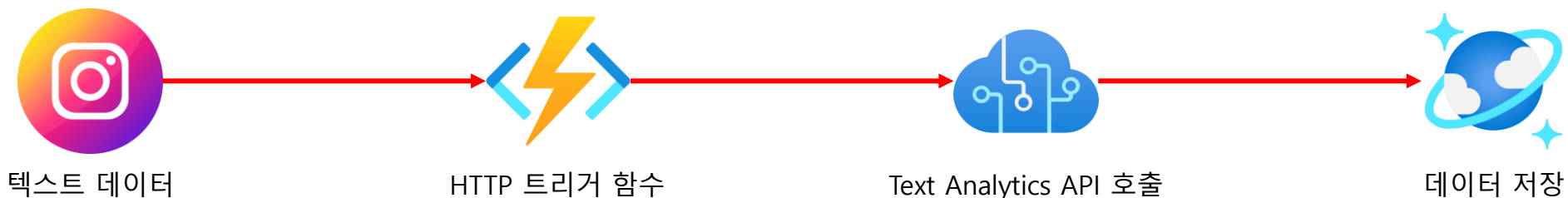
1. **파일 업로드:** 파트너 시스템에서 제품 카탈로그 정보가 담긴 파일을 Azure Blob Storage에 업로드
2. **Blob Storage 트리거:** Blob Storage에 파일이 업로드 되는 즉시, Blob Storage 트리거가 Azure Functions를 활성화 (예를 들어: 파일 업로드라는 "시작 버튼"을 누르는 것과 같습니다.)
3. **Azure Functions 실행 및 파일 처리:**
 - 트리거 된 Azure Functions는 업로드 된 파일을 다운로드 하여 데이터 유효성 검사를 수행
 - 유효성 검사를 통과한 데이터는 Azure Functions 내에서 데이터 변환 작업 수행 (예를 들어, 파일 형식 변환 (CSV to JSON), 데이터 정규화, 필요한 정보 추출 및 가공 등의 처리를 합니다.)
 - 변환된 데이터는 기본 시스템으로 전달하여 제품 카탈로그 정보를 업데이트를 진행 (데이터 베이스에 직접 저장하거나, API를 통해 다른 시스템과 연동할 수 있습니다.)
4. **처리 결과:** Azure Functions는 파일 처리 결과를 로깅하거나, 추가적인 후속 작업을 수행할 수 있습니다. 성공/실패 여부를 별도 저장소에 기록하거나 관리자에게 알림을 보내는 등의 기능을 구현할 수 있습니다.



실전 시나리오: AI 서비스와 연동

Azure Functions와 **Azure AI Services의 Text Analytics API**를 연동하여 **실시간 텍스트 감정 분석 시스템**을 구현하는 방법을 설명합니다.

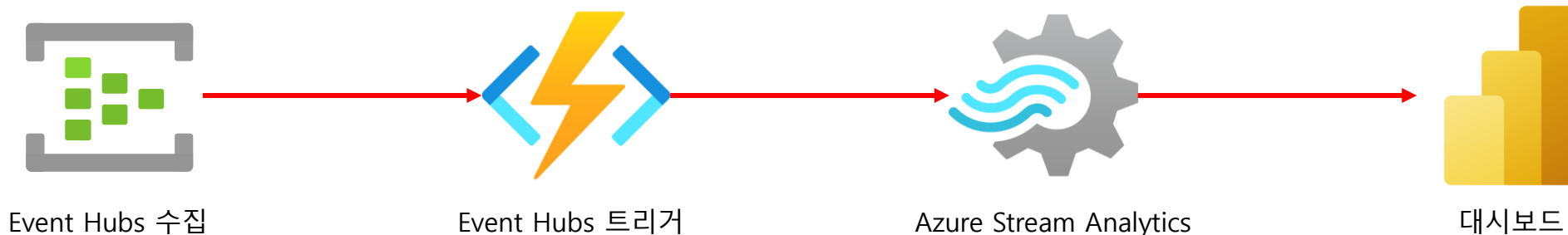
1. **텍스트 데이터 수집:** 웹 애플리케이션, 모바일 앱, 소셜 미디어 플랫폼 등 다양한 채널에서 수집되는 텍스트 데이터를 **HTTP 트리거**를 통해 Azure Functions로 전달
2. **HTTP 트리거 함수:** Azure Functions에서 **HTTP 트리거 함수**를 생성하여 텍스트 데이터 수집 요청을 처리
3. **Azure AI Services Text Analytics API 연동:** HTTP 트리거 함수 내에서 **Azure AI Services Text Analytics API**를 호출
 - API 호출 시, 분석할 텍스트 데이터를 API 요청 본문에 담아 전송하고, API 키 또는 인증 정보를 헤더에 포함하여 인증합니다.
 - Text Analytics API는 텍스트의 **감정 (긍정, 부정, 중립)** 과 함께 **신뢰도 점수**를 반환합니다. 더 나아가 **주요 구절**에 대한 감정 분석 결과도 얻을 수 있습니다.
4. **감정 분석 결과 처리 및 저장:** Text Analytics API로부터 감정 분석 결과를 수신한 후, Azure Functions는 다음과 같은 작업을 수행
 - **분석 결과 파싱:** API 응답에서 감정 분석 결과 (긍정/부정/중립, 신뢰도 점수 등)를 추출합니다.
 - **데이터 저장:** 분석 결과를 **Azure Cosmos DB, Azure SQL Database, Azure Storage** 등에 저장합니다. 데이터베이스 스키마는 분석 결과, 원본 텍스트, 메타데이터 등을 포함하도록 설계합니다.
 - **로그 기록:** 분석 요청 및 응답, 오류 정보 등을 로깅하여 시스템 모니터링 및 문제 해결에 활용합니다.



실전 시나리오: 실시간 데이터 처리

IoT 센서로부터 생성되는 방대한 양의 데이터를 실시간으로 처리하는 시나리오입니다.

1. **데이터 수집 (Event hubs):** 다수의 IoT 센서가 온도, 습도, 조도, 움직임 등 다양한 환경 데이터를 측정하여 **Event Hubs**로 전송
2. **Event Hubs 트리거:** 데이터가 도착하는 즉시, Azure Functions가 **Event Hubs 트리거**에 의해 자동으로 활성화 합니다.
(데이터 스트림의 유입을 감지하고, 데이터 처리 함수를 즉시 실행시키는 '자동 감지' 시스템으로 작동됨)
3. **Azure Function을 활용한 데이터 전 처리 및 분석:**
 - 트리거 된 Azure Functions는 **Event Hubs**로부터 센서 데이터를 받아 **전 처리** 과정을 수행합니다. **데이터 정규화, 오류 데이터 필터링, 필요한 정보 추출** 등 분석에 용이한 형태로 데이터를 가공합니다.
 - 전 처리된 데이터는 **Azure Stream Analytics**와 같은 실시간 데이터 분석 서비스로 전달되어 분석됩니다. IoT 센서 데이터 분석을 통해 의미 있는 정보를 실시간으로 추출할 수 있습니다. 예를 들어, 특정 임계 값을 초과하는 온도 데이터를 감지하여 경고를 발생시킬 수 있습니다.
4. **처리 결과 활용:** 분석 결과는 **대시보드**를 통해 시각화 하여 실시간 모니터링에 활용하거나, **알림 시스템**과 연동하여 이상 상황 발생 시 즉시 관리자에게 전송할 수 있습니다.



실전 시나리오: 예약 작업 자동화

금융 서비스의 고객 데이터베이스에서 **중복 항목을 제거하는 작업**을 일정에 따라 자동화하는 시나리오

- 1. 예약 작업 설정 (Timer 트리거):** 함수를 실행할 **시간 및 주기**를 정의합니다.
(예를 들어, "매일 새벽 2시 " 와 같이 금융 서비스 시스템에 부하가 적은 시간을 선택할 수 있습니다.)
- 2. Azure Functions 자동 실행:** 설정된 시간 또는 주기가 되면 Timer 트리거가 자동으로 Azure Functions를 실행합니다. 금융 서비스 담당자는 별도의 수동 작업 없이, 예약된 시간에 데이터베이스 정리 작업이 시작되는 것을 확인할 수 있습니다.
- 3. 중복 항목 제거 작업 수행:** Azure Functions는 금융 서비스 **고객 데이터베이스를 조회하여 중복된 고객 정보 항목**을 식별하고 제거합니다.
(예를 들어, 이름, 연락처, 주소 등을 기준으로 중복된 항목을 식별하고, 가장 최근에 업데이트된 정보를 기준으로 중복 항목을 정리)
- 4. 작업 결과 및 로깅:** Azure Functions는 중복 제거 작업 실행 결과를 **로깅**합니다. 제거된 중복 항목 수, 제거된 항목의 상세 정보, 작업 성공/실패 여부 등을 로그에 기록하여 데이터 정리 작업의 이력을 관리합니다.



예약 작업 설정



Timer 트리거



중복 항목이 제거된 DB

실전 시나리오: 확장 가능한 API 구축

사용자 등록 기능을 제공하는 API를 구축하는 시나리오입니다.

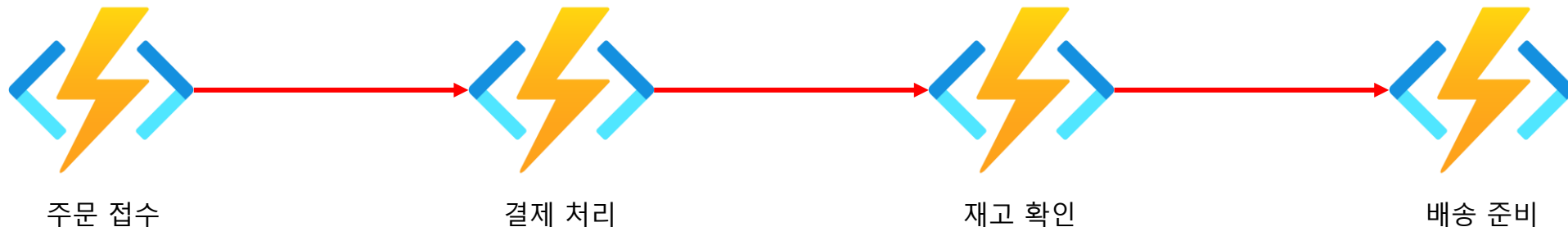
- 1. API 엔드 포인트 설계:** 사용자 등록에 필요한 API 엔드 포인트를 설계합니다. 각 엔드 포인트를 HTTP 메서드와 URL 경로를 통해 정의됩니다.
 - POST /api/register: **사용자 등록** – 신규 사용자 계정을 생성합니다.
- 2. HTTP 트리거 함수 구현:** 각 API 엔드 포인트별로 Azure Functions HTTP 트리거 함수를 작성합니다. 함수 내부에서는 사용자 인증 및 권한 부여 로직을 구현합니다.
- 3. 사용자 등록 (POST /api/register):**
 - 클라이언트 애플리케이션은 사용자 이름, 이메일, 비밀번호 등의 정보를 담아 /api/register 엔드 포인트로 POST 요청을 보냅니다.
 - Azure Function은 요청을 받아 다음과 같은 작업을 수행합니다:
 - 입력 데이터 유효성 검증 (필수 항목, 형식 검사 등)
 - 비밀번호 암호화 (단방향 해시 함수 사용)
 - 데이터베이스에 사용자 정보 저장
 - 성공적인 등록 완료 응답 (HTTP 201 Created, 성공 메시지 포함)



실전 시나리오: 복잡한 워크플로 구현 (Durable Functions) (1)

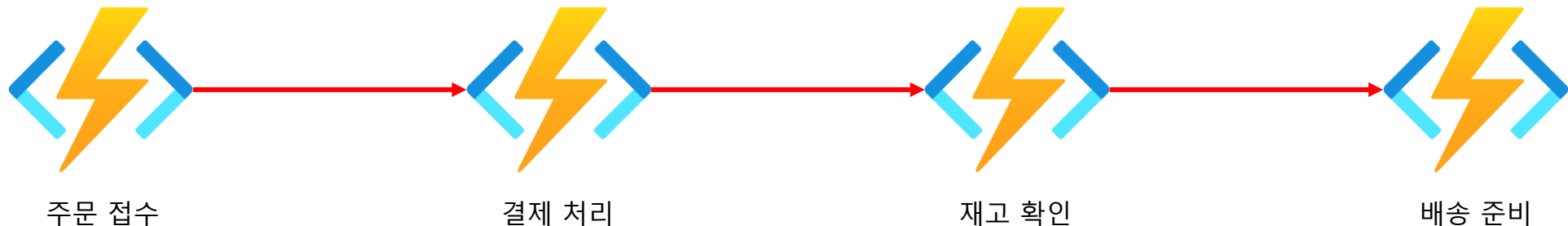
고객 주문 처리 워크플로를 구현하는 시나리오입니다. 운영 효율성을 높이고, 정확한 데이터 처리를 보장합니다.

- 주문 처리 워크플로 정의:** 주문 처리를 위한 Durable Functions 오케스트레이터 함수를 작성하여 전체 워크플로의 흐름을 정의합니다. 주문의 각 단계를 액티비티 함수로 호출하고, 단계 간의 순서와 병렬 처리를 제어합니다.
- 액티비티 함수 구현:** 주문 처리에 필요한 각 단계 (예: 주문 접수, 결제 처리, 재고 확인, 배송 준비 등)를 액티비티 함수로 작성합니다. 각 액티비티 함수는 특정 작업을 수행하며, 오케스트레이터 함수로부터 호출됩니다.
- 주문 접수:** 고객이 웹사이트에서 주문을 완료하면, 주문 데이터가 Azure Functions에 전달되어 주문 접수 액티비티 함수가 실행됩니다. 이 함수는 고객 정보를 확인하고 주문 세부 사항을 데이터베이스에 저장합니다.



실전 시나리오: 복잡한 워크플로 구현 (Durable Functions) (2)

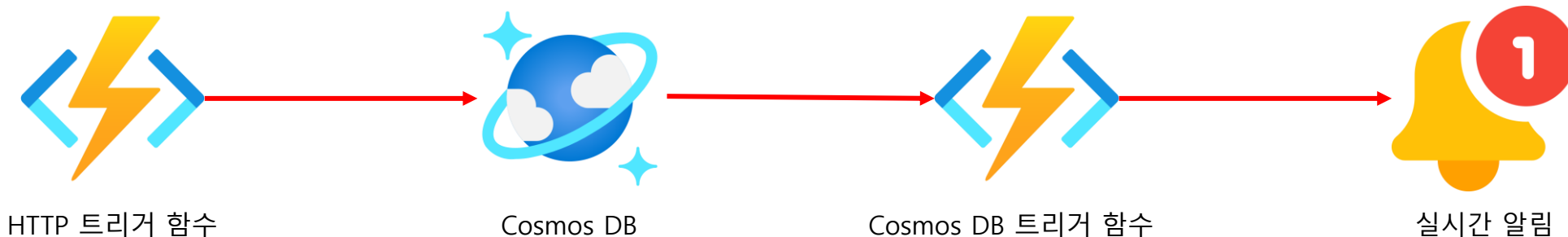
- 결제 처리:** 주문 접수 후, 결제 처리 액티비티 함수가 호출되어 결제 정보를 확인하고 결제를 승인합니다. 결제 실패 시, 오류 처리 로직을 통해 고객에게 알림 메시지를 전송합니다.
- 재고 확인:** 결제가 완료되면, 재고 확인 액티비티 함수가 호출되어 해당 상품의 재고를 확인합니다. 재고가 부족할 경우 고객에게 재고 부족 알림을 보내고, 처리를 중단합니다.
- 배송 준비:** 재고가 충분하다면, 배송 준비 액티비티 함수가 호출되어 주문 상품을 포장하고 배송 요청을 생성합니다. 이 단계에서 배송사의 API와 연동하여 배송 추적 정보를 업데이트합니다.
- 작업 결과 및 로깅:** 각 단계의 실행 결과를 로깅하여 추후에 주문 처리 상태를 확인할 수 있도록 합니다. 문제가 발생할 경우 관련 데이터를 저장하고, 관리자에게 알림을 전송하여 신속한 조치를 취할 수 있도록 합니다.



실전 시나리오: 데이터베이스 변경 감지 (1)

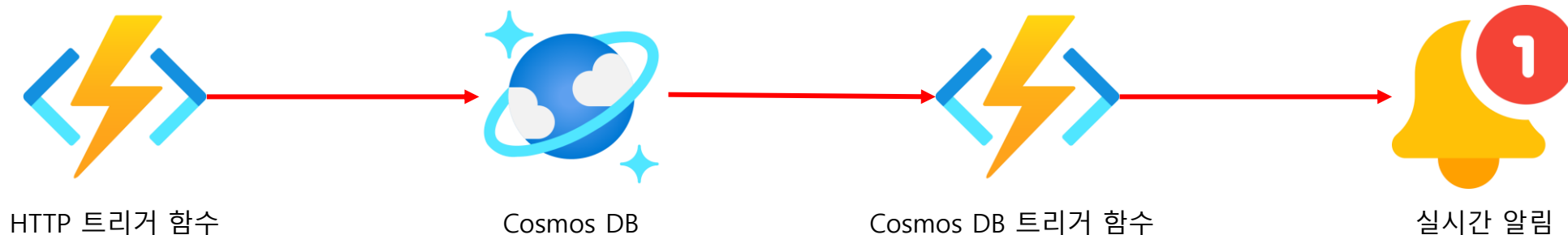
거래 내역 및 사용자 행동 로그를 분석하고 이상 거래를 감지하여 정기 감사를 수행하는 시스템을 구축하는 시나리오입니다. HTTP 트리거와 Cosmos DB 트리거를 조합하여 사용자 활동 로그를 수집하고, 데이터 변경 사항을 실시간으로 기록합니다.

1. **거래 기록 수집 (HTTP 트리거):** 고객이 온라인 플랫폼에서 거래를 완료할 때마다, 웹 애플리케이션 또는 API Gateway는 HTTP 요청을 Azure Functions로 전송합니다. 이 요청에는 거래 ID, 사용자 ID, 상품 정보, 거래 금액, 결제 수단, 거래 시간 등이 포함됩니다.
2. **HTTP 트리거 함수:** Azure Functions에서 거래 기록 HTTP 트리거 함수를 생성합니다. HTTP 요청을 수신하고, 요청 본문에 담긴 거래 데이터를 추출하여 Cosmos DB에 안전하게 저장합니다. 데이터는 감사 및 분석에 용이하도록 구조화된 형태로 저장됩니다.
3. **사용자 행동 로그 수집(HTTP 트리거):** 웹/앱 사용자의 로그인, 계정 정보 변경, 상품 조회, 장바구니 추가 등 주요 행동을 HTTP 트리거를 통해 Azure Functions에 전송하여 기록합니다. 사용자 행동 패턴 분석 및 이상 행위 탐지를 위한 기초 데이터를 확보합니다.



실전 시나리오: 데이터베이스 변경 감지 (2)

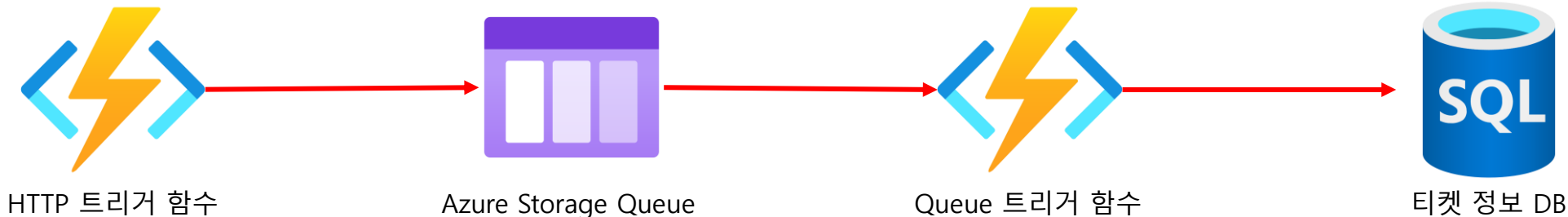
4. **이상 거래 감지 (Cosmos DB 트리거):** 거래 데이터베이스에 **새로운 거래 기록이 추가될 때마다** Azure Functions가 자동으로 실행되도록 합니다. 실시간 거래 감시 시스템을 구축하여 이상 거래를 즉시 탐지합니다.
5. **이상 징후 분석 및 탐지:** Cosmos DB 트리거 함수 내에서 **사전에 정의된 규칙 기반 분석** 또는 **머신 러닝 기반 이상 탐지 모델**을 활용하여 거래의 이상 징후를 분석합니다.
 - **규칙 기반 분석:** 비정상적으로 큰 금액의 거래, 특정 시간대에 집중된 거래 등 미리 정의된 규칙에 따라 이상 거래를 탐지합니다.
 - **머신 러닝 기반 탐지:** 정상 거래 패턴을 학습한 머신 러닝 모델을 사용하여 정상 범위를 벗어나는 거래를 이상 거래로 식별합니다.
6. **실시간 알림 및 알람:** 이상 거래가 감지되면 **관리자 또는 보안 담당자에게 실시간 알림**을 전송합니다. 이메일, SNS 등 다양한 채널을 통해 즉각적인 대응을 지원합니다.



실전 시나리오: 메시지 기반 시스템 구현

대량의 티켓 예매 요청을 효율적으로 처리하는 **티케팅 시스템**을 구현하는 시나리오입니다.

1. **사용자 티켓 예매 요청:** 사용자가 웹사이트나 모바일 앱을 통해 원하는 콘서트나 행사 티켓을 선택하고 예매 요청을 보냅니다.
2. **HTTP 트리거 함수:** Azure Functions의 **HTTP 트리거 함수**가 티켓 예매 요청을 접수합니다.
3. **큐에 메시지 저장:** HTTP 트리거 함수는 접수된 요청을 **Azure Storage 큐**에 메시지로 저장합니다. 큐는 예매 요청을 대기 열 형식으로 관리하여 비동기 처리를 가능하게 합니다.
4. **큐 스토리지 트리거 함수:** 큐에 새로운 메시지가 추가되면 **큐 스토리지 트리거 함수**가 자동으로 실행됩니다. 이 함수는 예매 요청을 처리하는 주요 로직을 담당합니다.
5. **티켓 처리 로직:** 큐 트리거 함수는 다음 작업을 수행합니다:
 - 티켓의 유효성 및 가용성 확인
 - 결제 정보 검증
 - 결제 성공 시 티켓 발급
 - 티켓 정보 데이터베이스 저장
 - 사용자에게 티켓 정보 이메일 발송
6. **작업 결과 로깅:** 모든 처리 결과를 로깅하여 예매 요청의 성공/실패 여부를 추적합니다. 문제 발생 시 운영팀에 즉시 알리를 보냅니다.



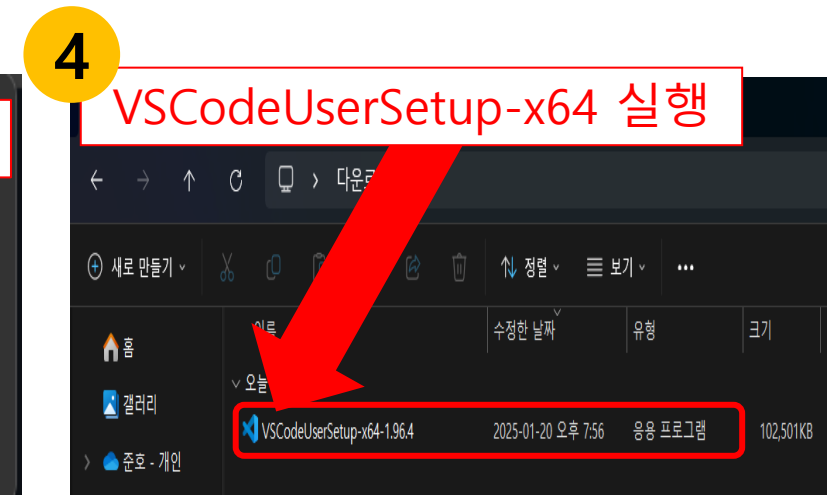
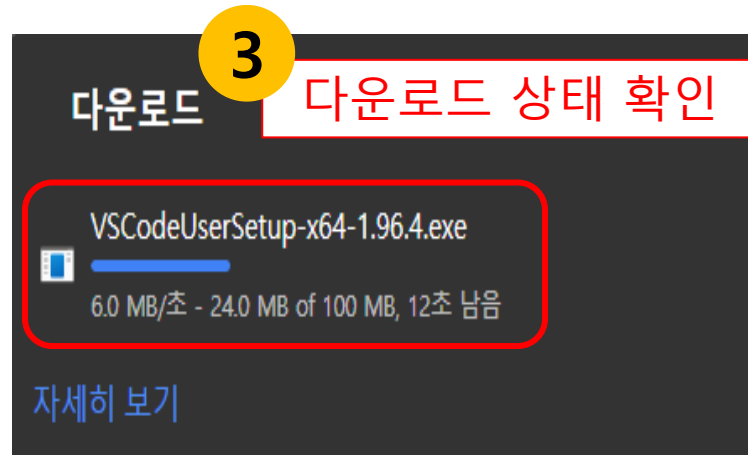
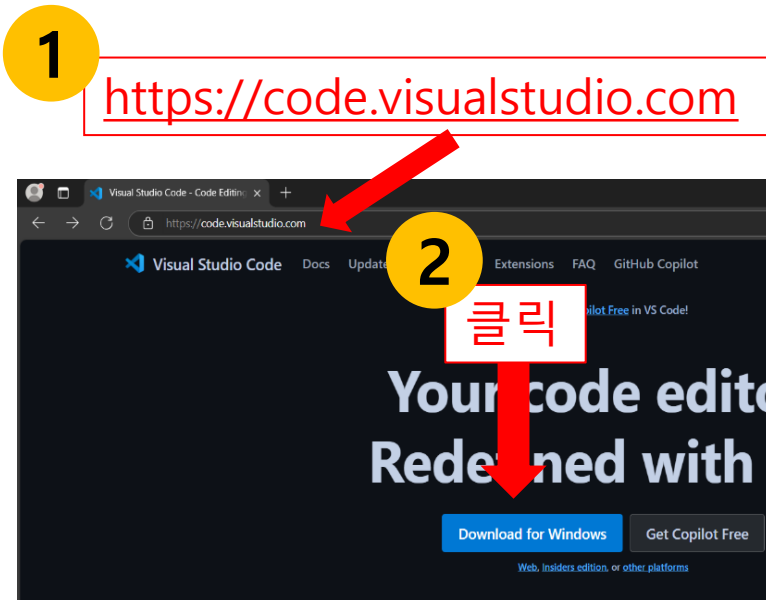
Azure Functions 시작을 위한 준비

Azure Functions 개발 환경 구축하기

VS Code 설치(1)

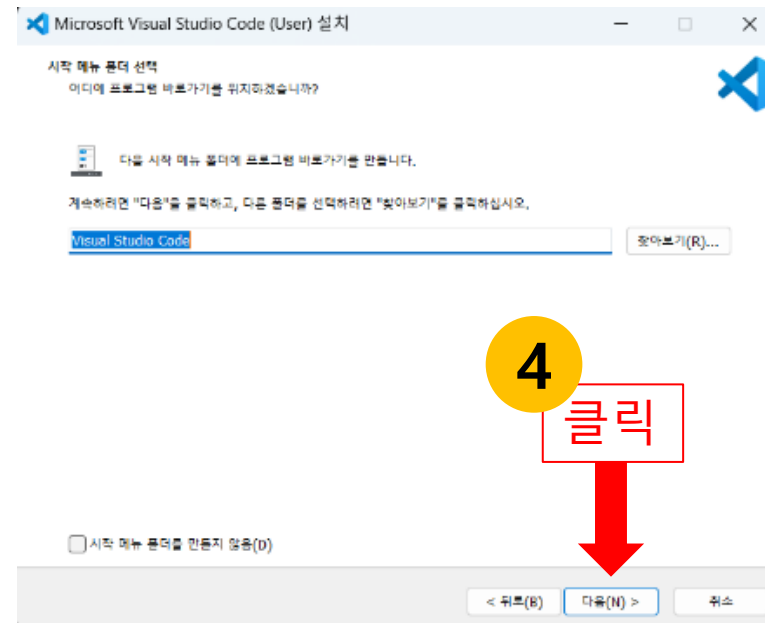
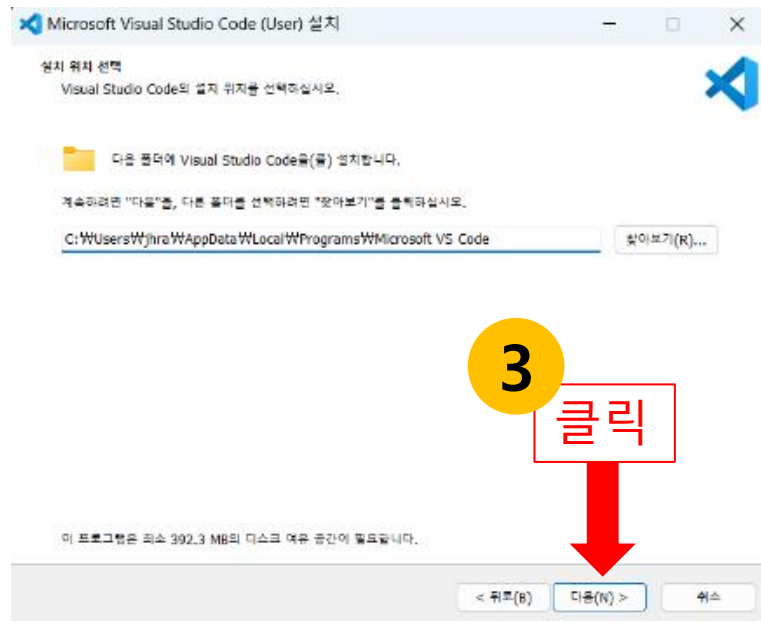
Azure Functions를 본격적으로 개발하기 전에, 개발환경을 먼저 준비해야 합니다. Azure Functions를 로컬 환경에서 실행하고 테스트하고, 작성한 코드를 쉽게 관리하려면 "VS Code" 라는 도구가 필요합니다.

1. VS Code 다운로드 페이지 접속
2. 운영체제에 맞는 설치 파일 선택
3. 설치 파일 실행: 다운로드 완료된 설치 파일 실행



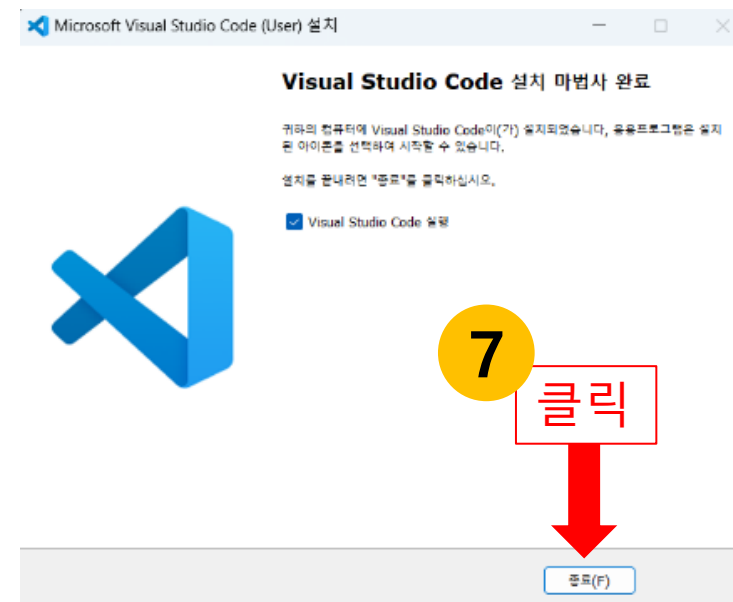
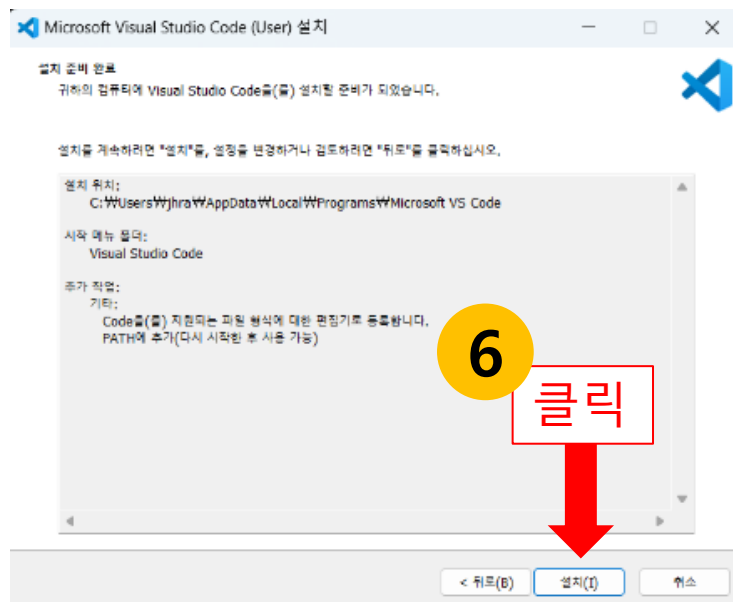
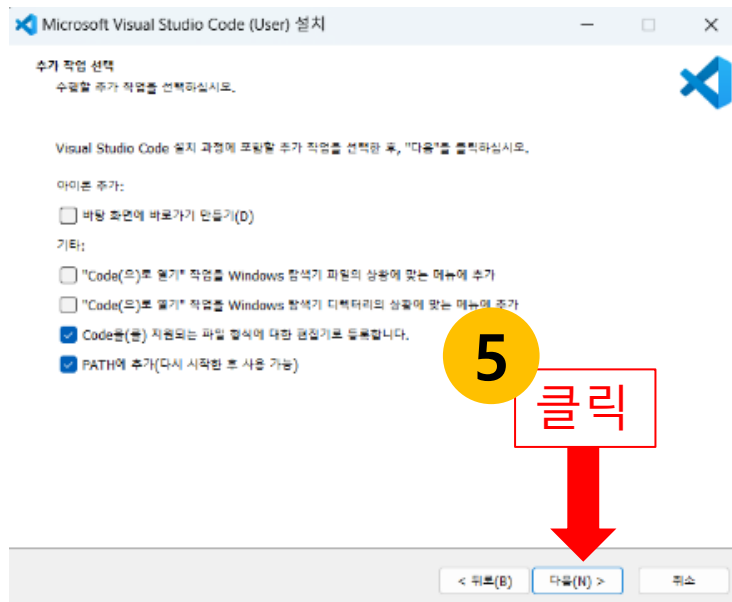
VS Code 설치(2)

4. 설치 진행: 화면 안내에 따라 다음 버튼 클릭



VS Code 설치(3)

5. "Visual Studio Code 설치 마법사 완료" 메시지 확인 후 "종료" 버튼 클릭

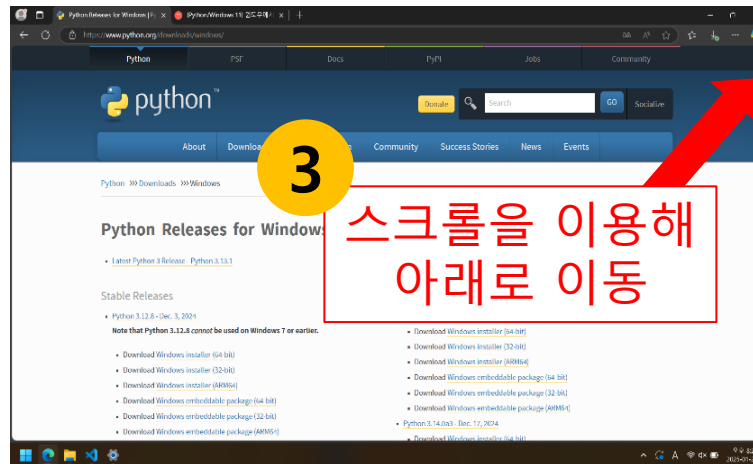
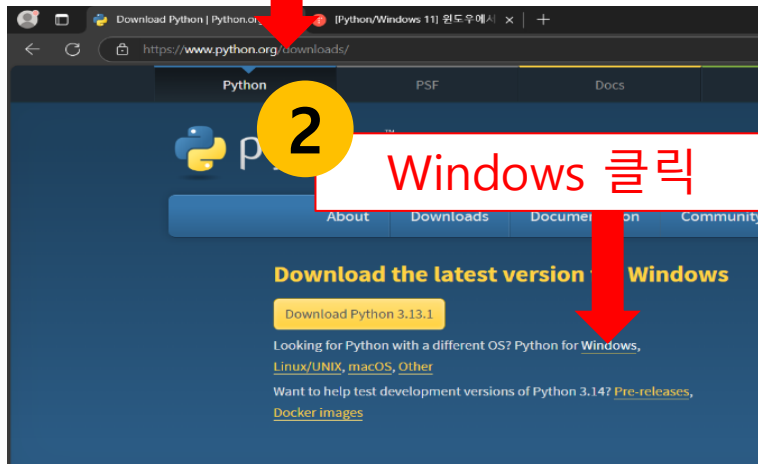


Python 설치(1)

Azure Functions와 Python을 함께 사용하면, 서버리스 환경에서 Python의 강력한 기능을 자유롭게 활용하여 다양한 종류의 함수를 효율적으로 개발할 수 있습니다.

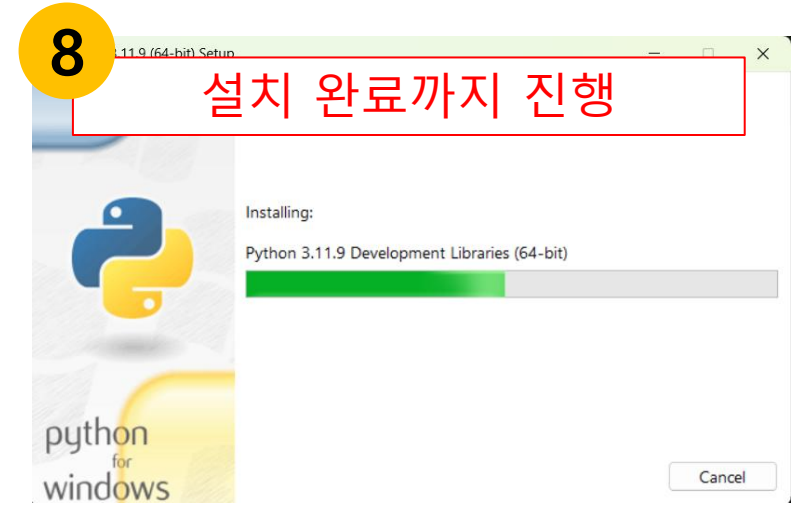
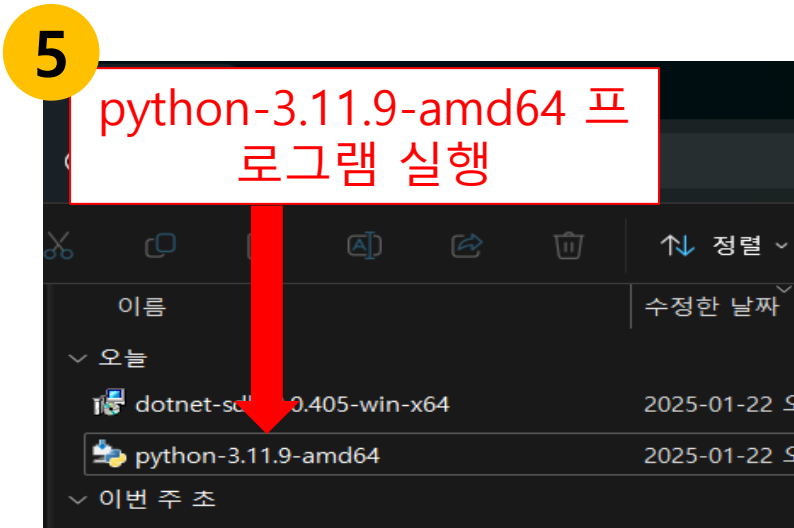
1. Python 다운로드 페이지 접속
2. Windows 버전 선택: 다운로드 페이지에서 "Windows" 버튼을 클릭합니다.
3. 설치 파일 다운로드: 페이지 아래로 스크롤하여 Python 3.11.9 버전 링크를 클릭하여 다운로드합니다.

1 <https://www.python.org/downloads>



Python 설치(2)

4. 설치 파일 실행: 다운로드 된 python-3.11.9-amd64.exe 파일을 실행합니다.
5. Add Python.exe to Path 체크 박스를 꼭 체크합니다.
6. Install Now 버튼을 클릭하여 기본 설정으로 설치를 진행합니다.
7. 설치 완료 확인: 설치 완료 메시지가 나타나면 Close 버튼을 클릭하여 설치를 완료합니다.



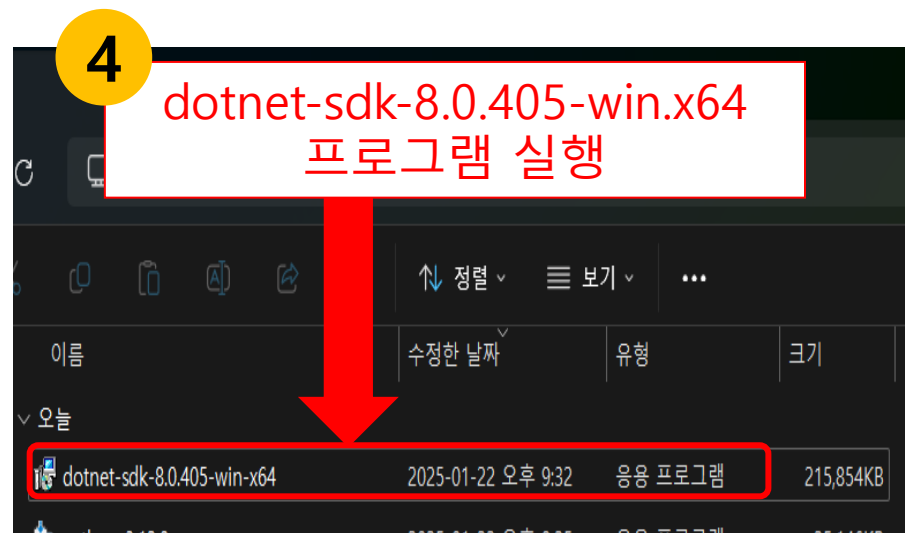
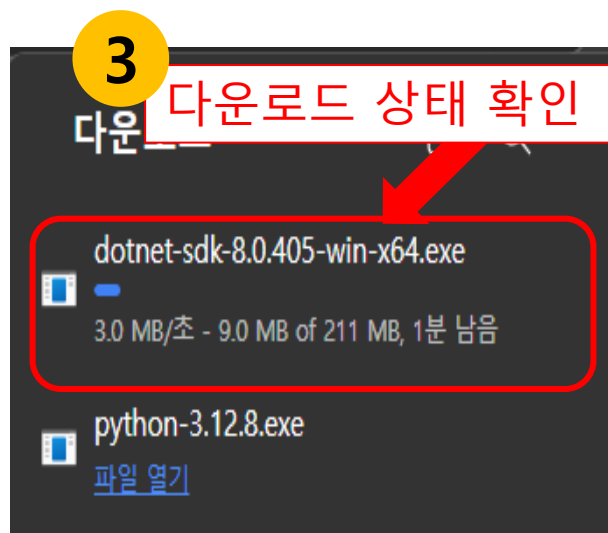
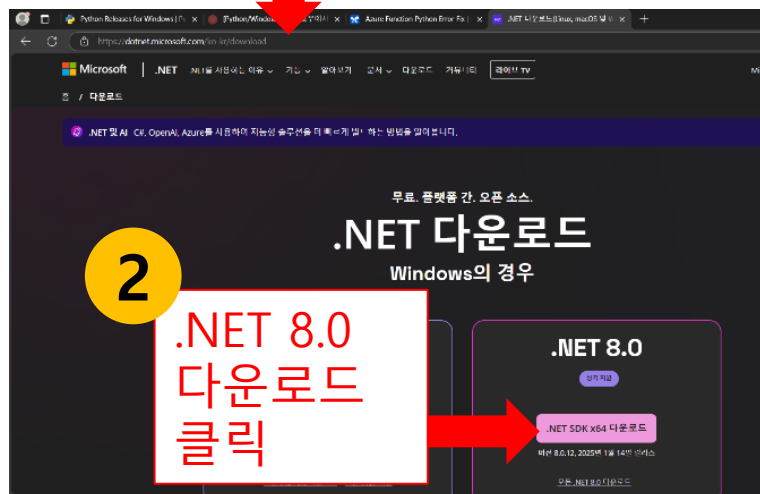
.NET SDK 설치(1)

.NET SDK는 Azure Functions를 개발하는 데 필수적인 도구로, 다양한 Azure 서비스를 간편하게 사용할 수 있는 기능과 라이브러리를 제공합니다.

1. .NET SDK 다운로드 페이지 접속
2. .NET 8.0 버전 선택: 페이지에서 .NET SDK 버전 다운로드 링크 클릭
3. 설치 파일 실행: 다운로드에 완료된 dotnet-sdk-8.0.405-win.x64 프로그램을 실행합니다.

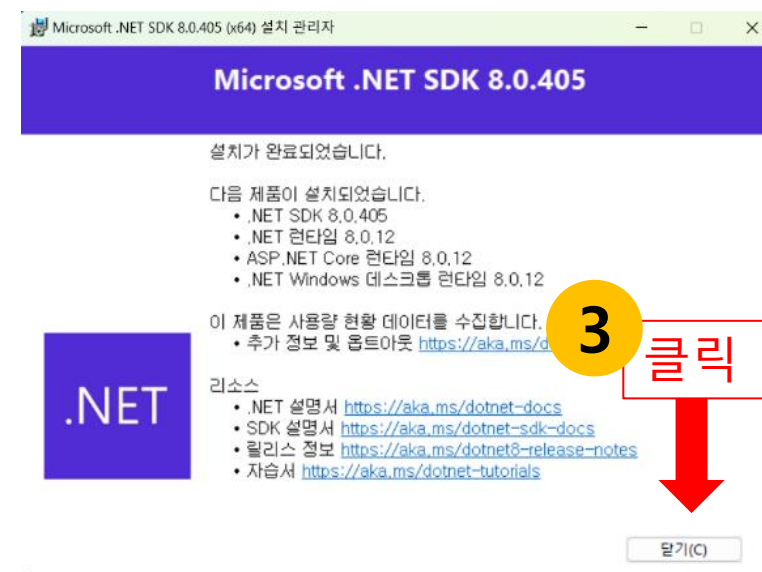
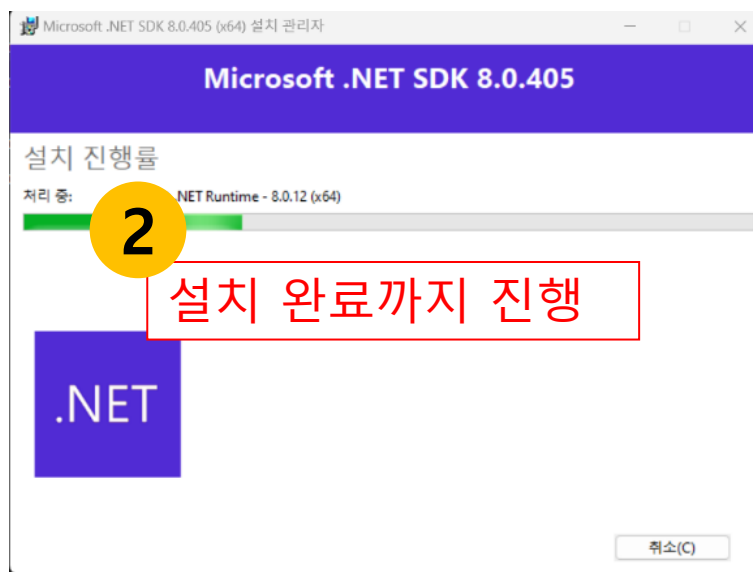
1

<https://dotnet.microsoft.com/ko-kr/download>



.NET SDK 설치(2)

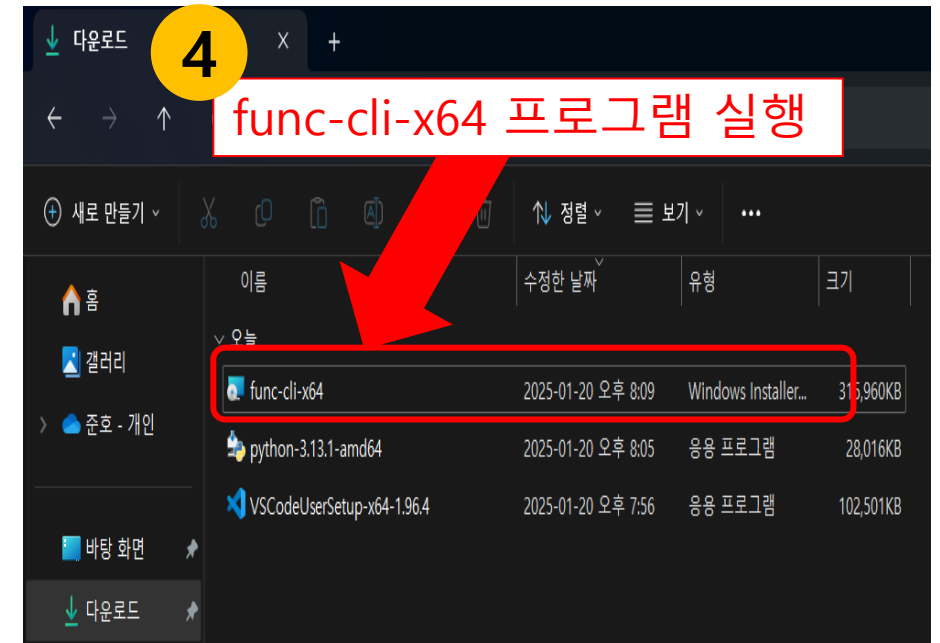
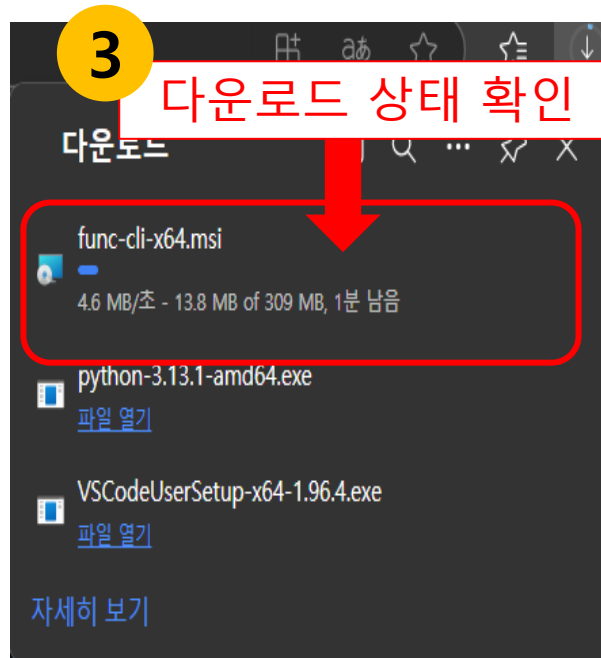
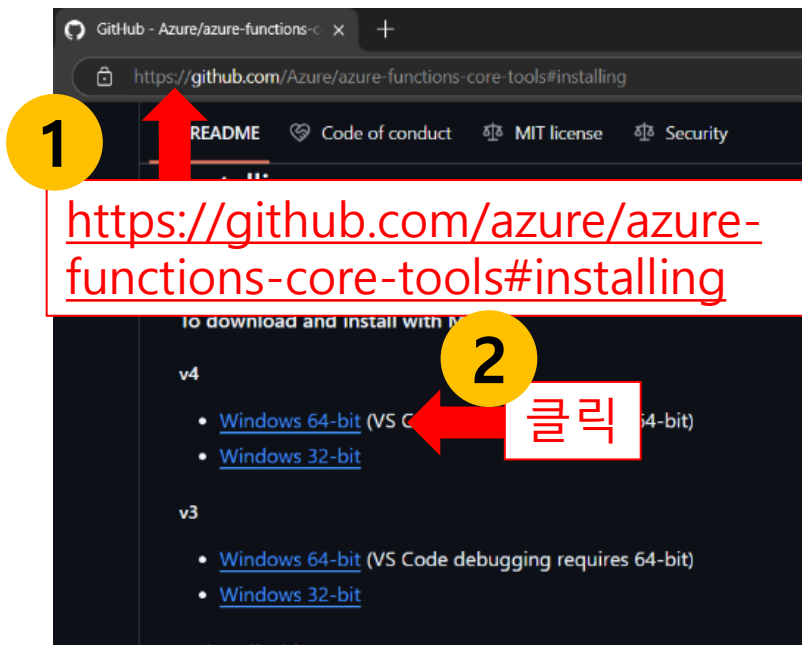
4. 설치 진행: 설치 프로그램의 안내에 따라 "설치" 버튼을 클릭합니다
5. "설치가 완료되었습니다." 메시지가 나타나면 "닫기" 버튼을 클릭하여 설치를 완료합니다.



Azure Functions Core Tools 설치(1)

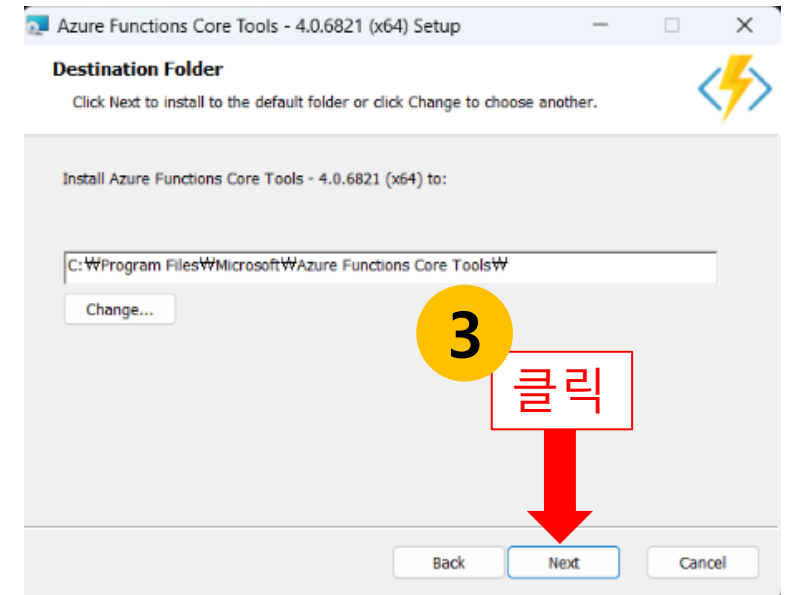
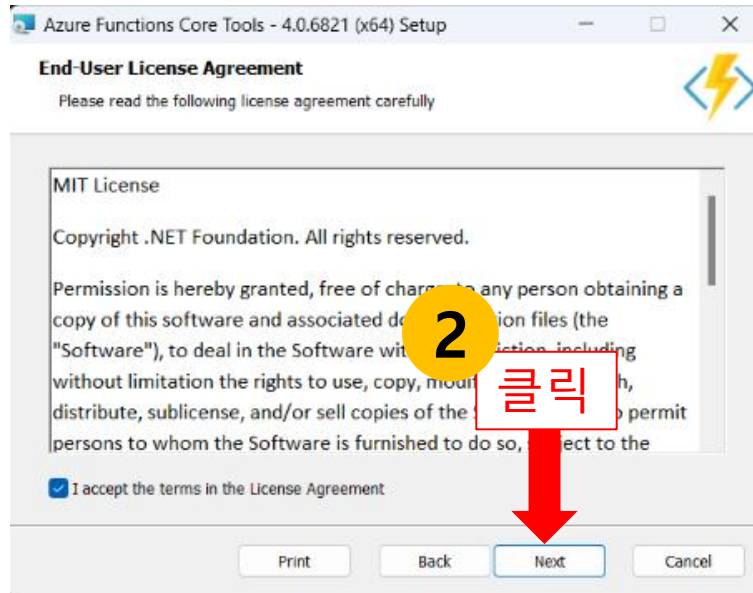
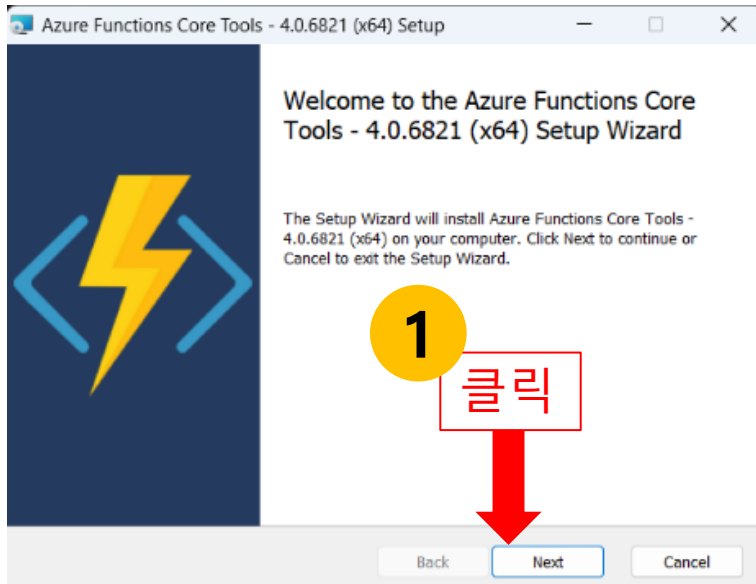
Azure Functions Core Tools를 설치하면, 인터넷 없이도 Azure Functions를 만들고, 실행하고, 디버깅하는 등 다양한 작업을 로컬 환경에서 편리하게 처리할 수 있습니다.

1. Azure Functions Core Tools 다운로드 페이지 접속
2. Windows 버전선택: 페이지에서 Windows 64-bit 버전 다운로드 링크 클릭
3. 설치 파일 실행: 다운로드에 완료된: func-cli-x64 프로그램을 실행합니다.



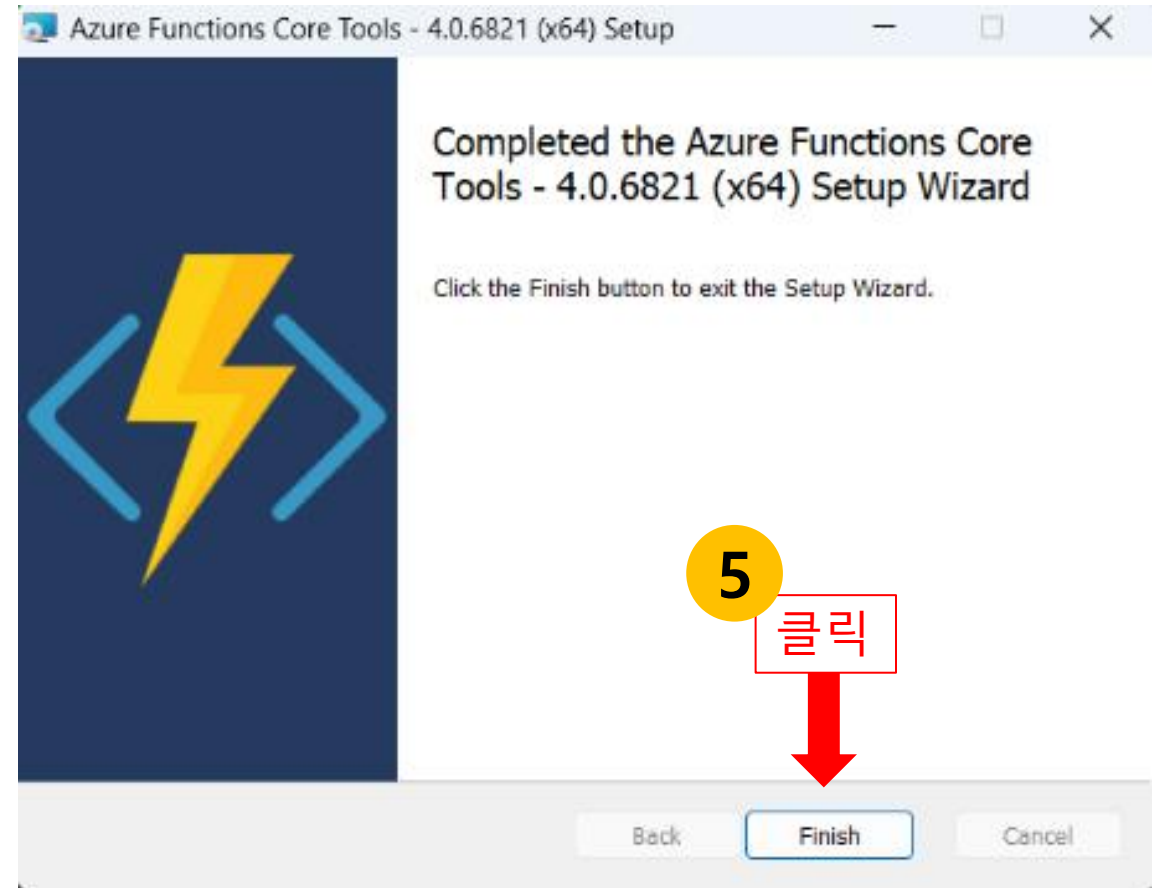
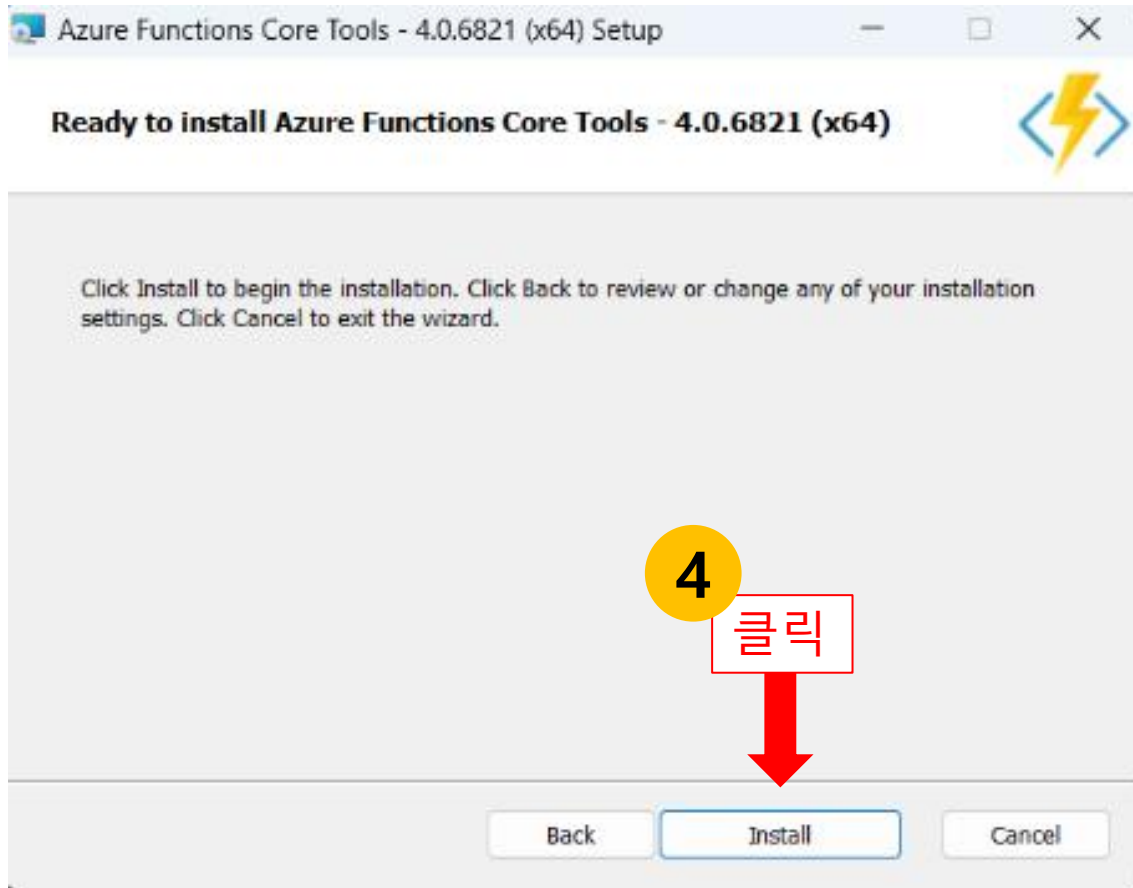
Azure Functions Core Tools 설치(2)

4. 설치 진행: 설치 마법사가 실행되면 "Next" 버튼을 클릭하여 설치를 진행합니다.



Azure Functions Core Tools 설치(3)

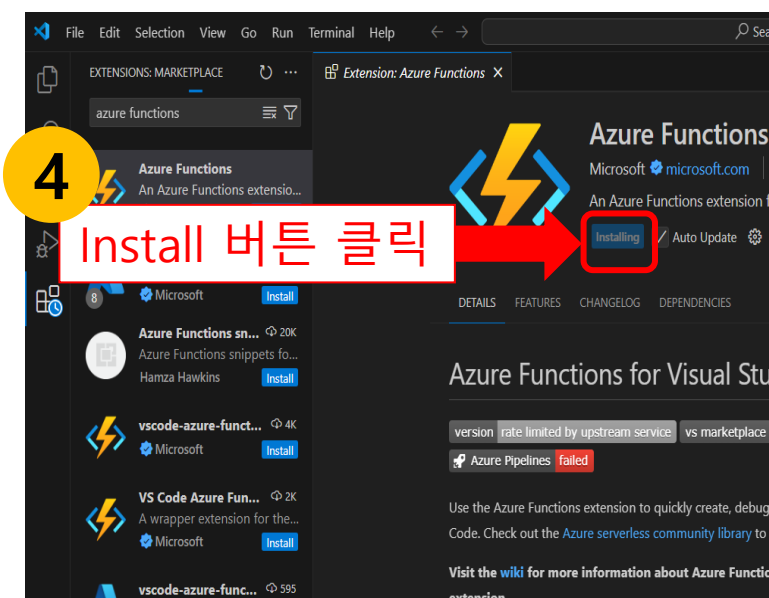
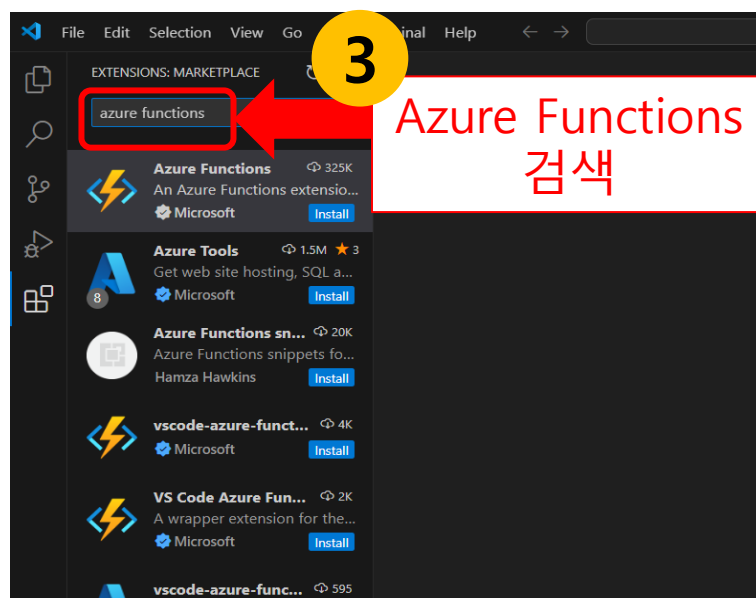
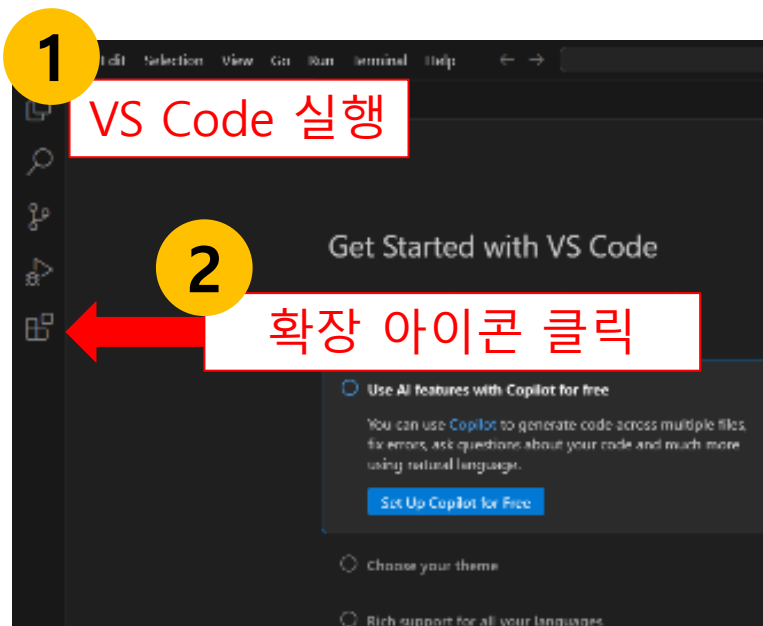
5. "Completed the Azure Functions Core Tools" 메시지가 나타나면 "Finish" 버튼을 클릭합니다.



Vs Code 확장 설치: Azure Functions

VS Code를 Azure Function 개발에 최적화된 환경으로 만들어 줍니다. Azure Functions 프로젝트 생성, 함수 추가, 로컬 디버깅, Azure 배포 등 모든 개발 과정을 VS Code 안에서 편리하게 처리할 수 있게 됩니다.

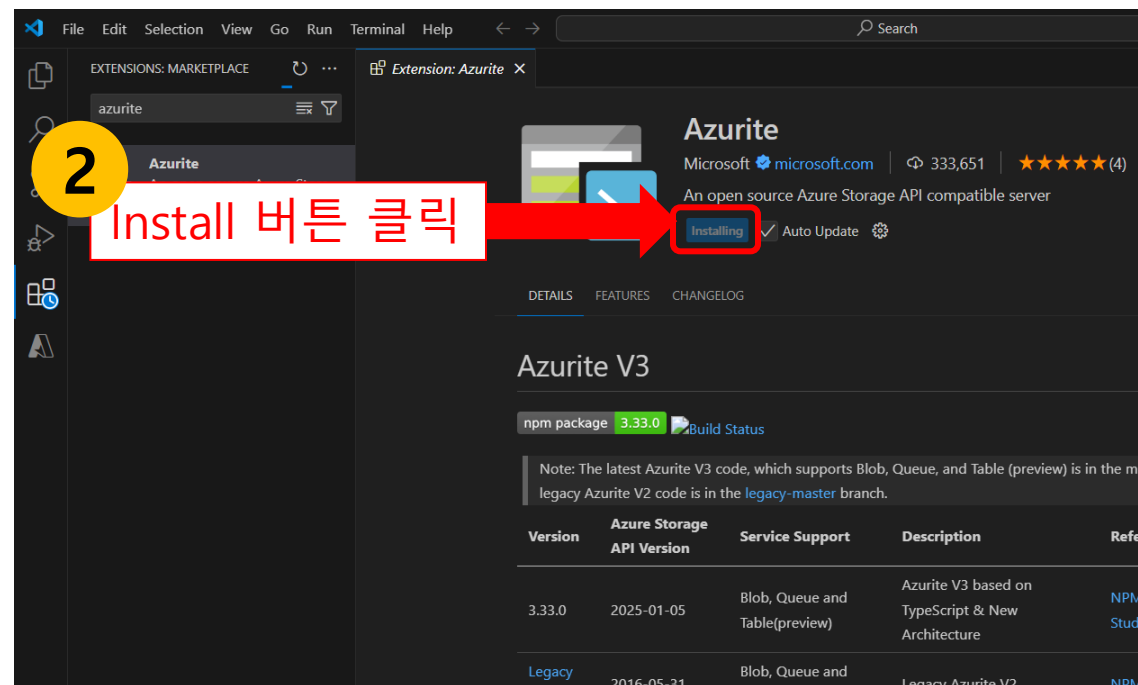
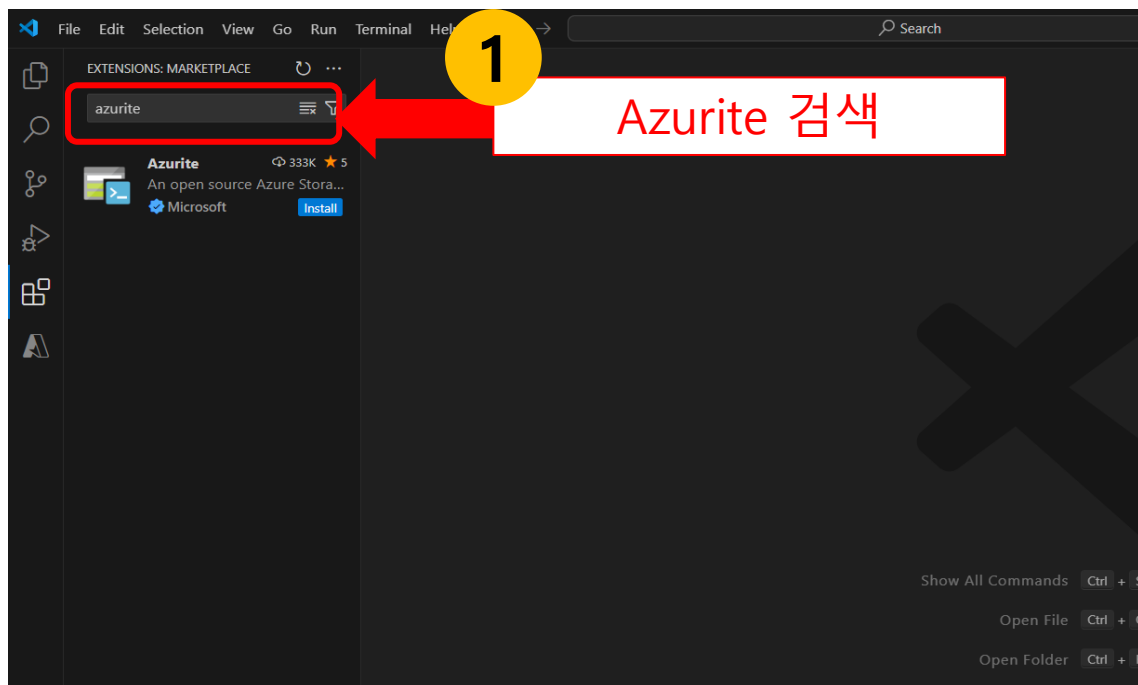
1. 확장 (Extension) 메뉴 이동: VS Code 왼쪽 사이드바에서 “확장” 아이콘을 클릭합니다.
2. Azure Functions 확장 검색: 확장 메뉴 상단 검색창에 “Azure Functions”를 입력합니다.
3. “Azure Function” 확장 설치: 검색결과에서 “Azure Function” 확장을 찾아 “Install” 버튼을 클릭합니다.



Vs Code 확장 설치: Azurite

Azurite는 Azure Storage 서비스의 로컬 구현체로, Azure Functions의 테스트 및 실제 Azure Storage 없이도 로컬 환경에서 데이터 저장과 처리를 가능하게 해주는 도구입니다.

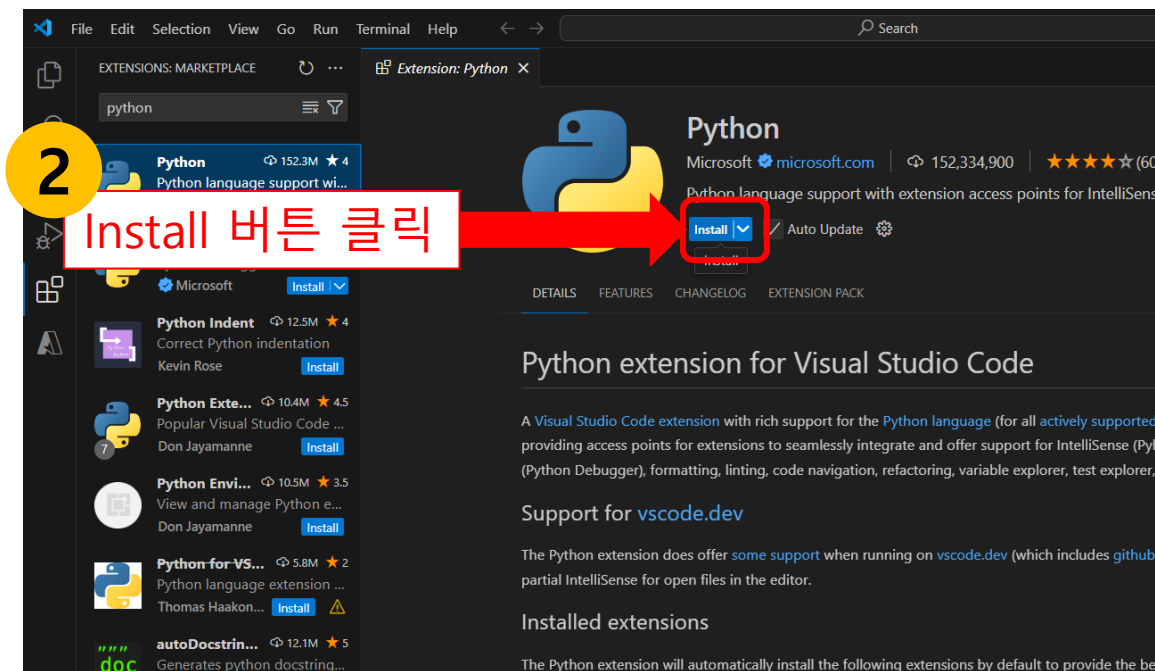
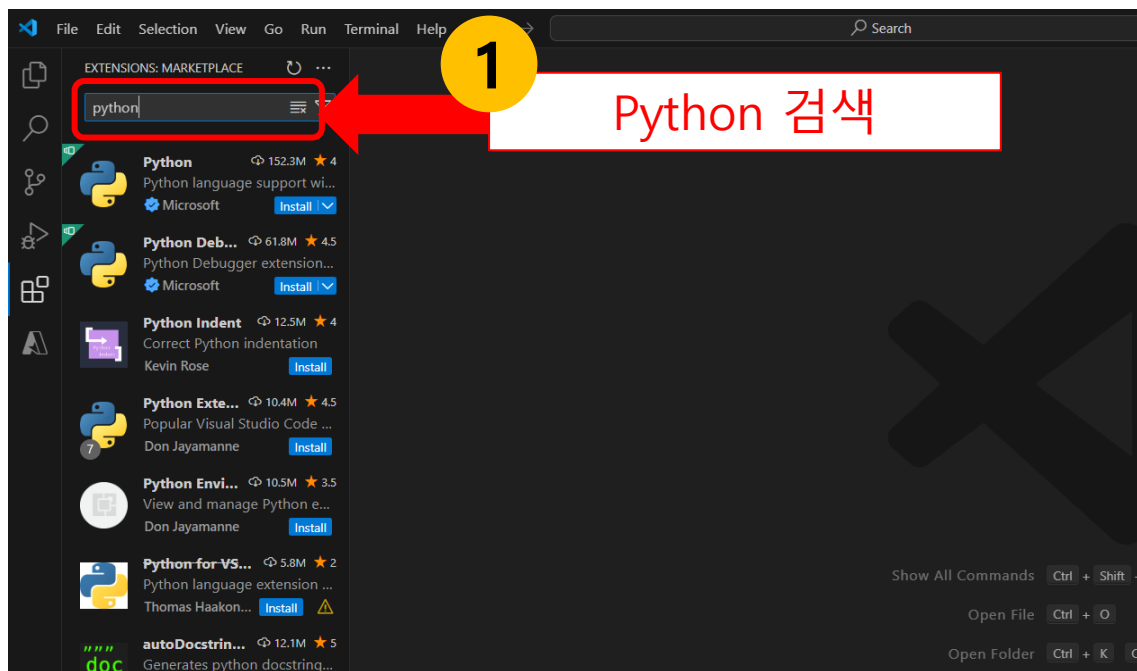
1. "Azurite" 확장 검색: 확장 메뉴 상단 검색 창에 "Azurite"를 입력합니다.
2. "Azurite" 확장 설치: 검색 결과에서 "Azurite" 확장을 찾아 "Install" 버튼을 클릭합니다.



Vs Code 확장 설치: Python

Python은 Azure Functions와 함께 할 때 더욱 강력한 성능을 발휘합니다. VS Code의 Python 확장을 설치하면 코드 편집, 실행, 디버깅을 더욱 간편하게 할 수 있게 됩니다.

1. "Python" 확장 검색: 상단 검색 상에 "Python"을 입력합니다.
2. "Python" 확장 설치: 검색 결과의 "Python" 확장 목록에서 "Install" 버튼을 클릭합니다.



수업 내용 요약

- Azure Function은 서버리스 컴퓨팅의 장점을 최대한 활용하면서 개발자가 코드 작성에만 집중할 수 있게 해주는 강력한 서비스입니다.
- 필요할 때만 리소스를 사용하고, 사용한 만큼만 비용을 지불하는 경제적인 모델을 통해 비즈니스 가치를 극대화할 수 있습니다.
- 다양한 트리거와 바인딩을 활용하면 클라우드 서비스 간의 통합을 손쉽게 구현할 수 있으며, 실시간 데이터 처리, 파일 업로드 자동화, AI 서비스 연동, 예약 작업 실행 등 다양한 비즈니스 요구사항을 효율적으로 해결할 수 있습니다.

감사합니다.

다음번 수업에서 만나요 😊